Internship Program

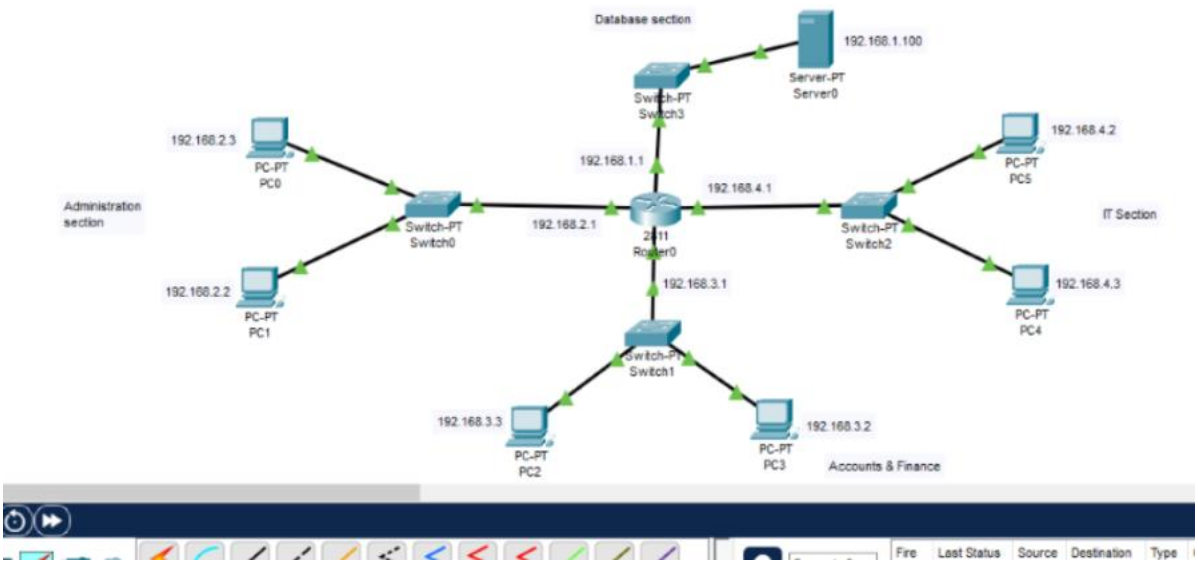
Mini Project

Submitted By: Maryam Rehan

Mentor: Miss Aroosa Tariq

OS & NetworkingFundamentals

Practical 01: Home/Office Network Design using Cisco Packet Tracer



Objective: To design and simulate a basic home/office network using Cisco Packet Tracer.

Tools Used: Cisco Packet Tracer

Procedure:

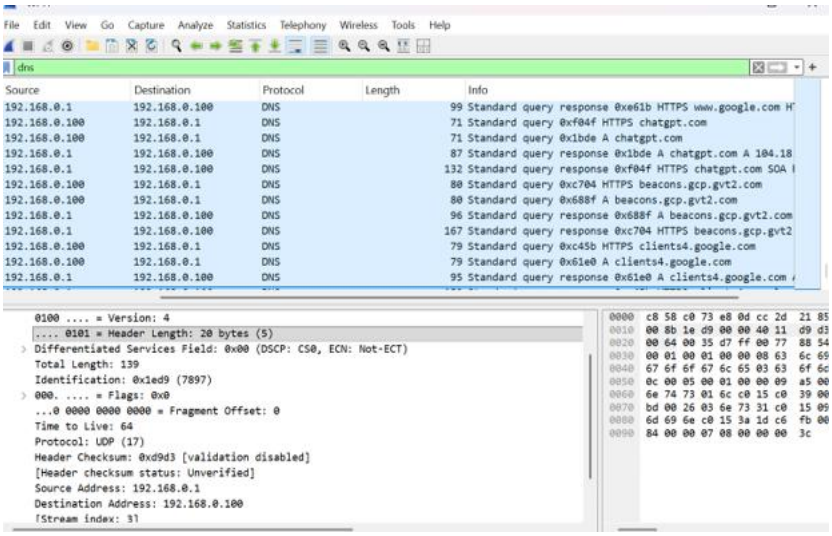
- Created a network topology with a router, switch, computers, laptop, and printer.
- Connected the network devices using appropriate connections.
- Assigned IP addresses to the devices.
- Configured the network to enable communication between connected devices.
- Tested connectivity using the ping command.

Result: Successfully designed a basic home/office network topology in Cisco Packet Tracer.

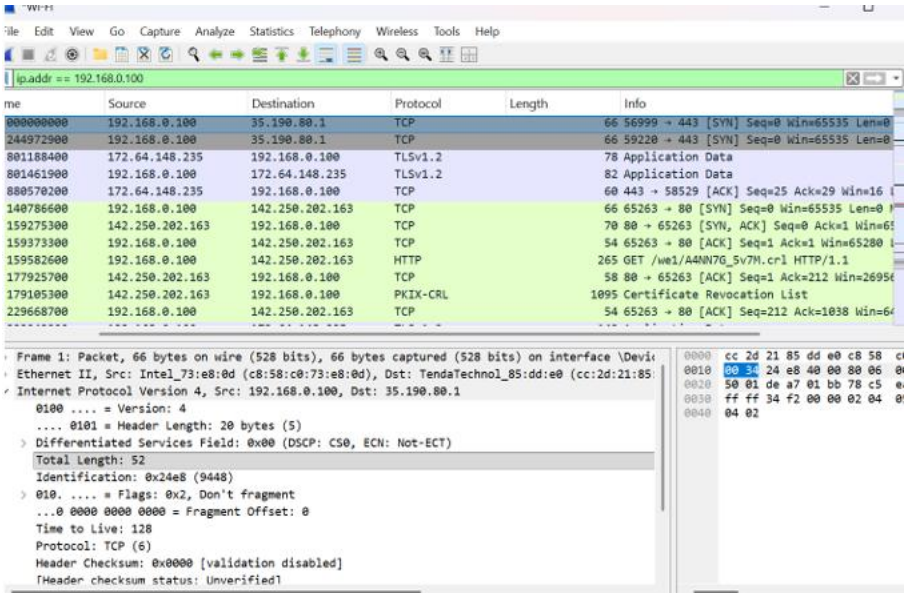
Learning Outcome: Learned the basics of network topology, IP addressing, device connectivity, and network communication.

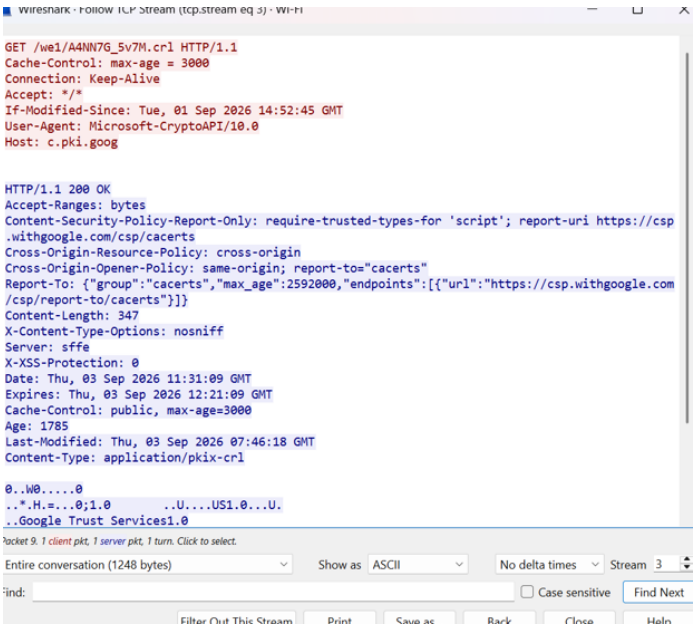
Practical 02: Capture and Analyze Network Traffic using Wireshark

DNS



TCP





Objective: To capture and analyze network traffic and identify TCP and DNS packets using Wireshark.

Tools Used: Wireshark and a web browser.

Procedure:

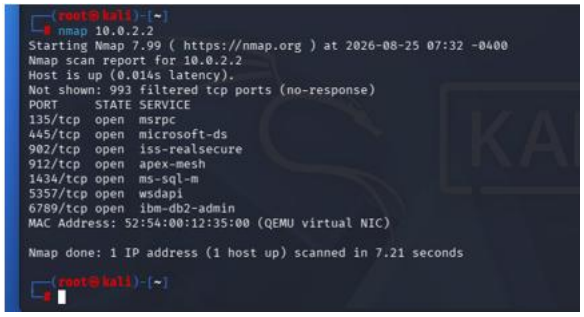
1. Opened Wireshark and selected the active network interface.
2. Started capturing live network traffic.
3. Generated network traffic by browsing websites.

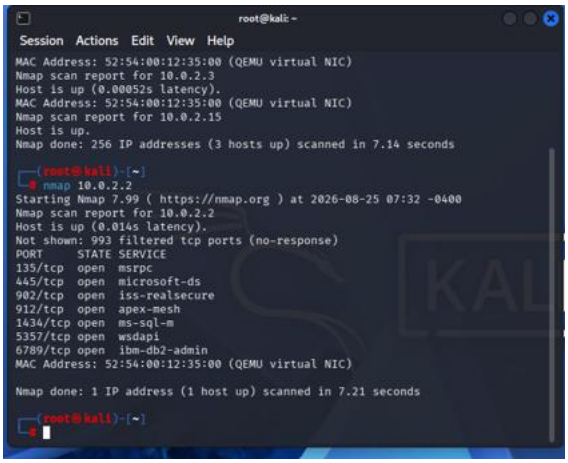
4. Applied the tcp display filter to identify TCP packets.
5. Applied the dns display filter to examine DNS queries and responses.

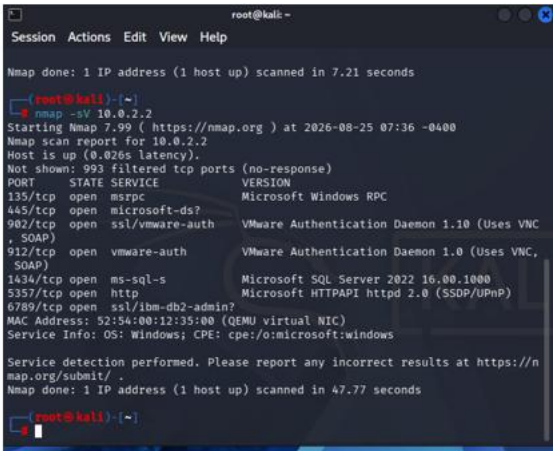
Result: Successfully captured network traffic and identified TCP and DNS packets using Wireshark filters.

Learning Outcome: Learned the basics of packet capture, protocol identification, and network traffic analysis.

Practical 03: Nmap Scan on Your Own Lab







Objective: To identify open TCP ports and detect running services on an authorized lab host using Nmap.

Tools Used: Nmap 7.99 and Kali Linux.

Procedure:

1. Executed a basic scan using the command nmap 10.0.2.2.
2. Identified the host status and open TCP ports.
3. Recorded the discovered ports, including 135, 445, 902, 912, 1434, 5357, and 6789.
4. Performed service version detection using nmap -sV 10.0.2.2.
5. Examined the service information returned by Nmap, including Microsoft RPC and VMware-related services.

Findings: The scan reported seven open TCP ports on the target host. Service version detection provided additional information about some of the services, including VMware Authentication Daemon on port 902.

Result: Successfully performed a basic Nmap scan and service version detection in the lab environment.

Learning Outcome: Learned how to identify open ports, recognize network services, and use Nmap for basic security assessment.

Practical 04: Python Packet Sniffer using Scapy

```
┌──(root㉿kali)-[~/demo]
└─# python3 --version
Python 3.13.12

┌──(root㉿kali)-[~/demo]
└─# python3 -c "import scapy; print(scapy.__version__)"
2.7.01

┌──(root㉿kali)-[~/demo]
└─# nano sniffer.py
```

```
┌──(root㉿kali)-[~/demo]
└─# ip -br addr
lo               UNKNOWN        127.0.0.1/8 ::1/128
eth0             UP             10.0.0.2/24 fe80::fc10:7c36:5c40:5dc2/64
docker0          DOWN           172.17.0.1/16
```

```
  GNU nano 9.0                         sniffer.py *
from scapy.all import sniff, IP, TCP, UDP, ICMP

def show_packet(packet):
    if IP in packet:
        src = packet[IP].src
        dst = packet[IP].dst

        if TCP in packet:
            protocol = "TCP"
        elif UDP in packet:
            protocol = "UDP"
        elif ICMP in packet:
            protocol = "ICMP"
        else:
            protocol = "Other IP"

        print(f"{protocol}: {src} → {dst}")

print("Sniffing on eth0...")
print("Press Ctrl+C to stop.")

sniff(iface="eth0", prn=show_packet, store=False)


^G Help        ^O Write Out   ^F Where Is    ^K Cut         ^T Execute
```

```
┌──(root㉿kali)-[~/demo]
└─# python3 sniffer.py
Sniffing on eth0...
Press Ctrl+C to stop.
```

Objective: To develop a basic Python packet sniffer to capture and display network packet information.

Tools Used: Python 3.13.12, Scapy 2.7.01, and Kali Linux.

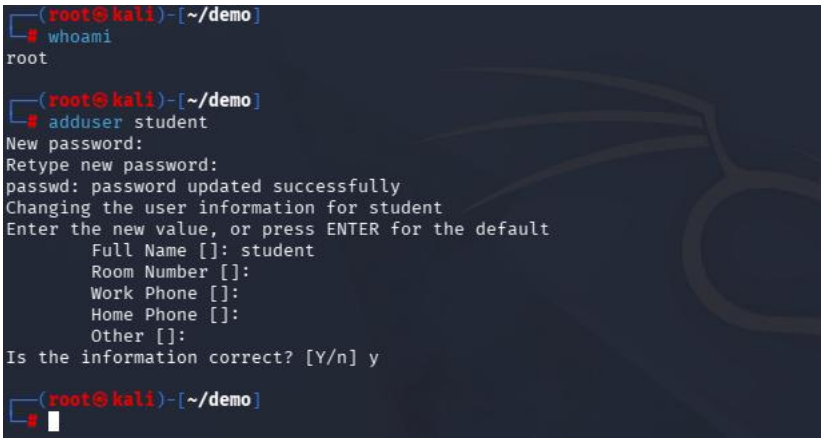
Procedure:

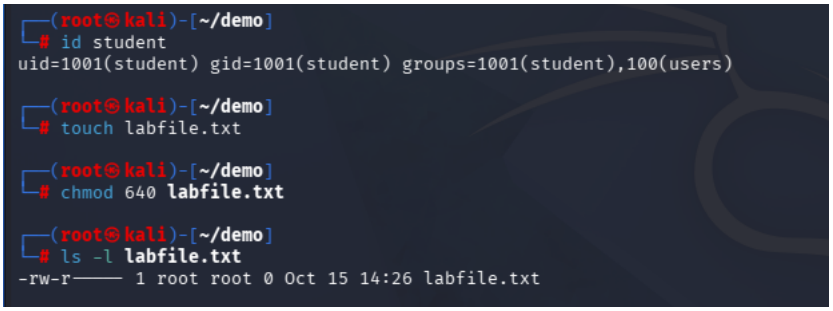
1. Verified the Python and Scapy installations.
2. Created a Python script named sniffer.py.
3. Used Scapy to capture IP packets and identify TCP, UDP, and ICMP protocols.
4. Displayed source and destination IP addresses.
5. Executed the script and tested it by generating network traffic.

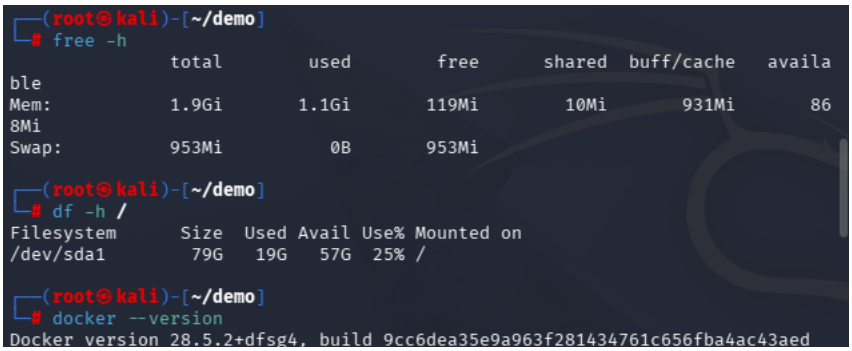
Result: Successfully executed the Python packet sniffer and prepared it to capture network packets.

Learning Outcome: Learned basic packet sniffing, protocol identification, and network traffic monitoring using Python and Scapy.

Practical 05 Linux VM with User Permissions and Wazuh SIEM Lab







```
┌──(root㉿kali)-[~/demo]
└─# systemctl status docker --no-pager
● docker.service - Docker Application Container Engine
     Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset:
 enabled)
     Active: active (running) since Thu 2026-10-15 12:47:51 EDT; 1h 42min ago
 Invocation: a0e0ef34964649abae83275fd839f079
TriggeredBy: ● docker.socket
       Docs: https://docs.docker.com
   Main PID: 688 (dockerd)
      Tasks: 10
     Memory: 109.1M (peak: 119.3M)
        CPU: 5.805s
     CGroup: /system.slice/docker.service
             └─688 /usr/sbin/dockerd -H fd:// --containerd=/run/containerd/c…

Oct 15 12:47:49 kali dockerd[688]: time="2026-10-15T12:47:49.836813318-0…ory"
Oct 15 12:47:49 kali dockerd[688]: time="2026-10-15T12:47:49.837129188-0… ... "
Oct 15 12:47:50 kali dockerd[688]: time="2026-10-15T12:47:50.848737468-0…und"
Oct 15 12:47:50 kali dockerd[688]: time="2026-10-15T12:47:50.851066484-0…ne."
Oct 15 12:47:51 kali dockerd[688]: time="2026-10-15T12:47:51.241705670-0…fsg4
Oct 15 12:47:51 kali dockerd[688]: time="2026-10-15T12:47:51.244563936-0…kit"
Oct 15 12:47:51 kali dockerd[688]: time="2026-10-15T12:47:51.338341766-0…ion"
Oct 15 12:47:51 kali dockerd[688]: time="2026-10-15T12:47:51.405564663-0…ion"
Oct 15 12:47:51 kali dockerd[688]: time="2026-10-15T12:47:51.406352280-0…ock"
Oct 15 12:47:51 kali systemd[1]: Started docker.service - Docker Applica…ine.
Hint: Some lines were ellipsized, use -l to show in full.
```

```
┌──(root㉿kali)-[~/demo]
└─# docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES

┌──(root㉿kali)-[~/demo]
└─# free -h
               total        used        free      shared  buff/cache   availa
ble
Mem:           1.9Gi       1.0Gi       113Mi        10Mi       982Mi        91
2Mi
Swap:          953Mi       4.0Ki       953Mi
```

Objective: To practice Linux user management and file permissions, and understand the role of Wazuh SIEM in security monitoring.

Tools Used: Kali Linux, Linux Terminal, Docker.

Procedure:

1. Created a Linux user named student.
2. Verified the account using the id command.
3. Created a file named labfile.txt.
4. Applied file permissions using chmod 640 labfile.txt.
5. Verified file permissions using ls -l.
6. Checked Docker service status and available system memory in preparation for SIEM deployment.

Result: Linux user creation and file permission configuration were completed successfully. Docker was running, but no containers were active. Wazuh deployment and alert verification remain pending due to limited available memory.

Learning Outcome: Learned basic Linux account management, file access permissions, and the purpose of SIEM tools in security monitoring.



